From Voice to Text and Back

ASR, CTC, WER and text-to-speech analysis of a speech sample with disfluencies



Sample profile

English MP3 | 37.64 seconds | 44,100 Hz

Wav2Vec2-base-960h + Whisper-base | mono ASR input

Executive summary

Purpose, scope, and outcomes

Purpose. This project walks through a speech technology pipeline using a short English recording with reported speech disfluencies. It combines acoustic inspection, two pretrained ASR systems, a CTC decoding demonstration, word-error-rate calculation, and speech synthesis. It is an educational engineering demonstration, not a diagnostic evaluation.

Activity objectives

01 Inspect audio
Visualize the waveform and log-Mel spectrum.

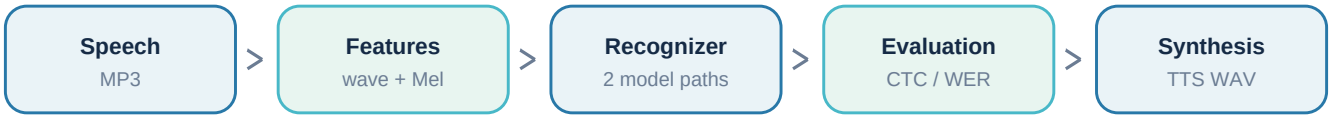
02 Recognize speech
Compare Wav2Vec2 with CTC and Whisper.

03 Explain decoding
Show CTC collapse/repeat and blank handling.

04 Evaluate
Calculate WER against a reference transcript.

05 Synthesize
Convert a recognized transcript back to audio.

End-to-end workflow



The experiment's two recognizers are parallel alternatives. Only the Wav2Vec2 path uses CTC; Whisper generates tokens with an encoder-decoder. A single recording and an unverified reference limit what can be concluded from the current scores.

Recording and data profile

Input characteristics and handling

Source recording

Property	Observed project value
File	audio/new_stutter.mp3
Duration	37.64 seconds
Decoded sample rate	44,100 Hz
Analysis channels	Mono for visual analysis and ASR
ASR rate	16,000 Hz for Wav2Vec2; Whisper decodes the file directly
Primary topic	Speaker describes daily life and difficulty speaking

Preprocessing choices. The plotting script loads the original sample rate and converts to mono. Wav2Vec2 uses librosa to resample to 16 kHz, its expected input rate. Whisper's audio decoder handles the MP3 and resampling internally. Keep the same source recording when comparing models.

Ethical and scope notes

Speech recordings and their transcripts can be sensitive personal data. Obtain informed permission for recording, processing, and redistribution, and share only what the speaker has authorized. The report describes observable pipeline behavior; it does not infer diagnosis, severity, or a person's identity.

Interpretation boundary

A pause or repeated acoustic pattern in a plot is not, by itself, proof of a stutter. Clinical conclusions require appropriate expertise and assessment.

Waveform analysis

Amplitude plotted against time

What the waveform shows

The waveform represents the audio signal's amplitude over time. It helps locate broad pauses, loudness changes, and regions with speech activity. It is a useful first inspection but does not encode lexical content and cannot reliably distinguish a filled pause, a stop closure, silence, or a recording artifact without listening and further analysis.

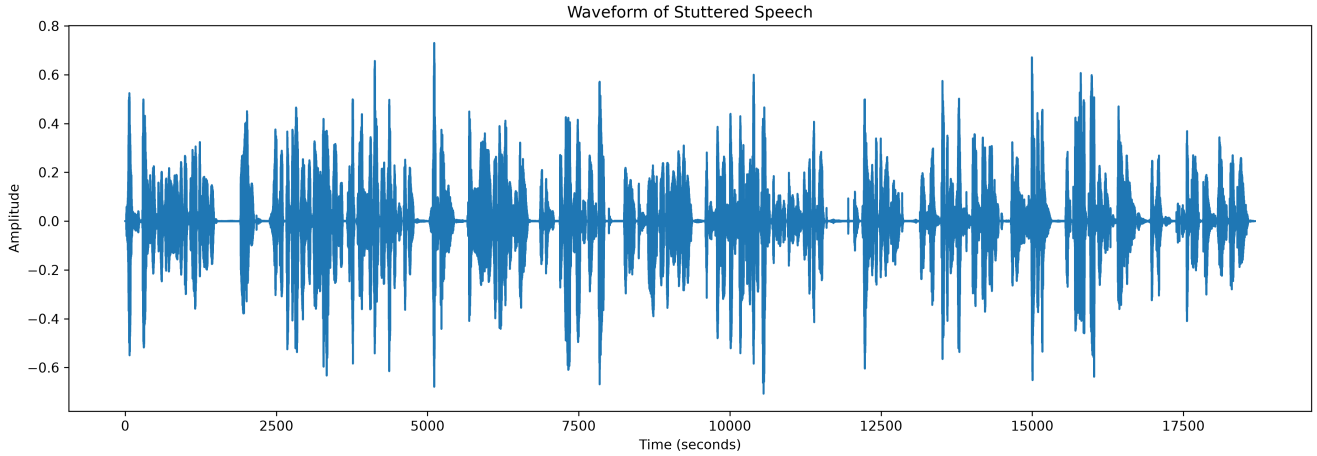


Figure 1. Waveform generated from the project recording (37.64 s, 44,100 Hz source rate). Time is shown horizontally and normalized audio amplitude vertically.

Feature extraction and log-Mel spectrum

From waveform samples to a time-frequency view

Log-Mel visualization

Short-time Fourier analysis decomposes local audio frames into frequency components. Mel filters combine the spectrum into perceptually spaced bands; conversion to decibels compresses the dynamic range and makes weaker components visible. This representation highlights spectral structure over time, rather than a single amplitude trace.

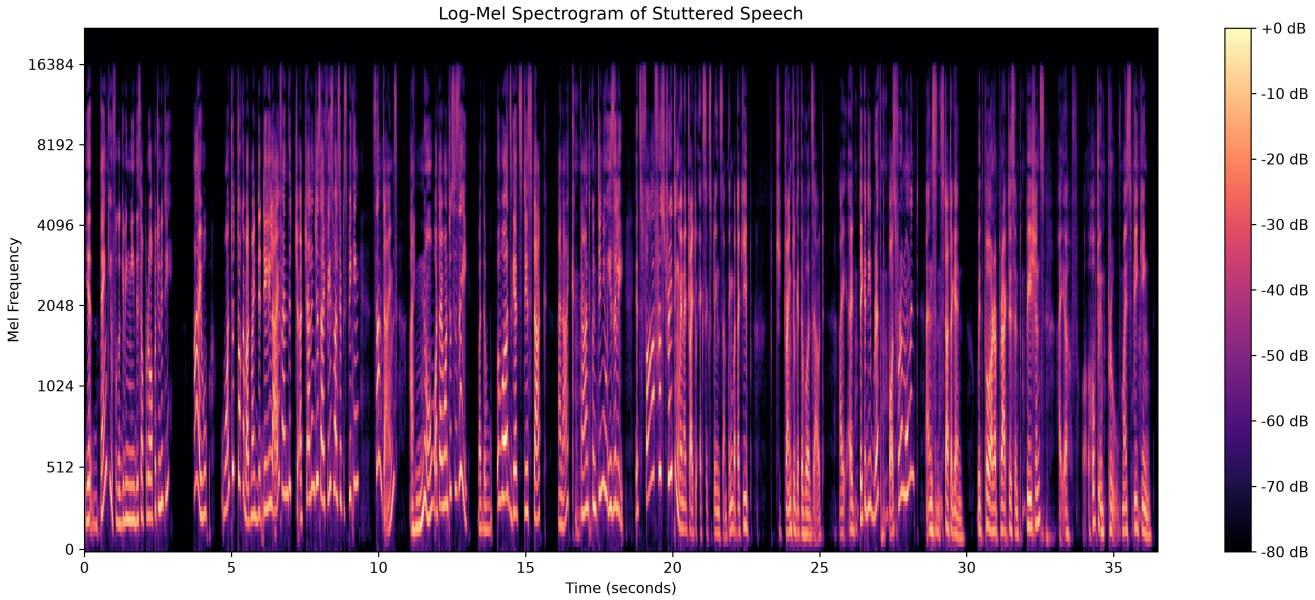


Figure 2. Log-Mel power spectrogram saved by main.py. Brighter regions indicate stronger relative spectral energy; frequency is displayed on the Mel scale.

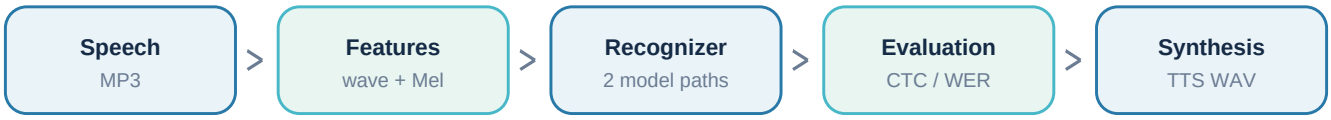
Feature parameters

Parameter	Value	Role
Mel bands	128	Frequency bins in perceptual scale
FFT window	2048 samples	Local spectral analysis window
Hop length	512 samples	Frame advance and time resolution
Reference level	Maximum power	Relative decibel scaling

ASR architecture

Two recognizer branches, one evaluation/synthesis path

Pipeline architecture



Branch	Input representation	Recognition / decoder
Wav2Vec2	16 kHz mono waveform	Context encoder › character logits › CTC
Whisper	Decoded audio / log-Mel features	Encoder › autoregressive text decoder
Shared evaluation	Reference + hypothesis text	Normalized words › edit counts › WER
TTS path	Selected transcript	Text synthesizer › generated speech waveform

Keep the model distinction clear

CTC is the Wav2Vec2 decoding method demonstrated here. Whisper is an encoder-decoder system and does not pass through the project's CTC decoder. They are not sequential components of one recognizer.

Wav2Vec2 ASR

Self-supervised speech representations with CTC output decoding

Model and inference

Checkpoint: facebook/wav2vec2-base-960h. The project uses the matching Wav2Vec2 processor and ForCTC model. Mono audio is resampled to 16 kHz and passed as input_values. The model returns frame-wise logits; argmax selects the most likely token per frame; processor.batch_decode applies CTC collapsing and converts tokens to text.

Recognized output (excerpt)

YE THANK YOU FOR GIVING ME THIS OPERA OPPORTUNITY I WANT TO TALK ABOUT MY DAILY LIFE AND THE CACHALLENGES I FACE WHILE SIP SPEAKING SOMETIMES I KNOW WHAT I WANT TO TSAY BUT THE WORDS GETS DISTUCK AND I CAN'T CONTINUE EASILY IT'S FAR FROM STRATING WHEN I TRY TO EX EXPLAIN SOMETHING BUT I KEEP SHE TRYING TO GET THE WORDS OUT I I TAKE A DEEP BREATH PAUSE FOR A MOMENT AND SINCE STURT AGAIN ...

Observation

The current result includes substitutions and inserted/repeated-looking words. That is plausible in spontaneous and disfluent speech, but each suspected error must be compared to the actual audio. A recognizer may also normalize or omit repetitions, so its output is not a verbatim ground truth.

CTC decoding explained

Connectionist Temporal Classification and the blank token

Why CTC is useful

During training/inference, an acoustic encoder produces a sequence of frame-level token distributions whose length is much greater than the transcript. CTC sums over valid alignments that collapse to the same target, avoiding the need to label an exact time for every character. Greedy decoding is a simple approximation that selects the best token independently at each frame.

Greedy collapse example

Frame path	blank | A | A | blank | A | B | B | blank
Merge adjacent duplicates	blank | A | blank | A | B | blank
Remove blank symbols	A | A | B
Decoded string	**AAB**

Why the blank matters

A A with no blank collapses to A. A blank separates repeated labels: A, blank, A becomes AA. The correct order is to merge adjacent repeats first, then remove blanks.

Whisper ASR

Encoder-decoder transcription with token generation

Inference setup

Implementation: faster-whisper, base model, CPU device, int8 compute, English language hint, and beam size 5. Whisper's audio processing creates log-Mel features internally and its decoder generates tokens conditioned on the encoded audio and previous tokens. It is architecturally different from Wav2Vec2-CTC.

Recognized output (excerpt)

Yeah, thank you for giving me this opportunity. I want to talk about my daily life and the challenges I face while speaking sometimes. I know what I want to say, but the words get stuck and I can't continue easily. It's far frustrating when I try to explain something but I keep trying to get the words out. I take a deep breath, pause for a moment and start again with poor practice and patience. I hope to improve my speaking.

What this output represents

This is a model-generated hypothesis, not a verified transcript. Fluent-looking punctuation and wording can conceal missed disfluencies or recognition errors. Always compare the text with the waveform and recording, especially when evaluating speech patterns.

WER methodology

Quantifying word-level transcription differences

Definition

Word error rate

WER = (S + D + I) / N. S = substitutions; D = deletions; I = insertions; N = number of reference words.

How the project computes it

Step	Operation
1. Reference	Load the local human/reference transcript file.
2. Hypotheses	Read Wav2Vec2 and Whisper transcript output files.
3. Normalize	Lowercase and remove punctuation consistently.
4. Align	JiWER aligns token sequences and counts edit operations.
5. Report	Print S/D/I and WER; save outputs/wer_results.csv.

Interpretation

WER is sensitive to the reference transcript and tokenization conventions. A value of 0.0 means exact normalized word agreement on this reference; a value above 1.0 is possible when insertions are numerous. Punctuation is typically excluded, but decisions about repetitions, partial words, and filled pauses must be consistent and documented.

ASR comparison results

Observed scores on this one recording and provisional reference

Current result table

Model	Hits	Sub.	Del.	Ins.	Ref words	WER
Wav2Vec2 CTC	74	7	0	8	81	18.52%
Whisper	81	0	0	1	81	1.23%

WER visual comparison



On the current transcript pair, Wav2Vec2 yields 7 substitutions and 8 insertions (15 edits / 81 words). Whisper yields 1 insertion (1 edit / 81 words). This is a descriptive result for this example only.

Critical caveat

The reference closely follows the Whisper output and has not been independently verified. That creates substantial evaluation bias. The 1.23% Whisper WER is not a fair benchmark and must not be interpreted as proof of accuracy or general superiority.

Qualitative output analysis

Differences between recognizer hypotheses

Side-by-side excerpts

Wav2Vec2 hypothesis

YE THANK YOU FOR GIVING ME THIS OPERA OPPORTUNITY I WANT TO TALK ABOUT MY DAILY LIFE AND THE CACHALLENGES I FACE WHILE SIP SPEAKING SOMETIMES I KNOW WHAT I WANT TO TSAY BUT THE WORDS GETS DISTUCK AND I CAN'T CONTINUE EASILY IT'S FAR FROM STRATING WHEN I TRY TO EX EXPLAIN SOMETHING BUT I KEEP SHE TRYING TO GET THE WORDS OUT I I ...

Whisper hypothesis

Yeah, thank you for giving me this opportunity. I want to talk about my daily life and the challenges I face while speaking sometimes. I know what I want to say, but the words get stuck and I can't continue easily. It's far frustrating when I try to explain something but I keep trying to get the words out. I take a deep breath, ...

What the current comparison can and cannot show

The output texts differ in recognition choices and in how fluency and punctuation are rendered. Such differences can reflect model design, training data, decoding, and normalization. The current WER calculation compares each output to a provisional text resembling Whisper; it does not establish which words were actually spoken. A defensible analysis requires an independently prepared reference and manual adjudication of ambiguous segments.

Recommended transcript protocol

Listen to the source audio; transcribe verbatim; define a consistent convention for repetitions, partial words, and fillers; have a second reviewer resolve uncertain spans; then calculate WER for both systems.

Text-to-speech output

Completing the return path from text to audio

Synthesis procedure

The PowerShell script reads outputs/whisper_transcription.txt and uses Windows System.Speech to synthesize the recognized text. The generated audio is saved as outputs/tts_output.wav. The notebook also offers an optional gTTS path, which requires network access.

Artifact	Observed properties
Generated file	outputs/tts_output.wav
Encoding	PCM WAV
Sample rate	22,050 Hz
Channels	1
Duration	28.70 seconds
Text source	Whisper transcript

Interpretation

Synthetic voice is not the source speaker

TTS demonstrates text-to-speech conversion only. It does not reproduce the source speaker's vocal identity, prosody, or stuttering. The output is a separate synthesized performance of the transcript.

Reproducibility and project structure

How to rerun the pipeline

Core files

File	Purpose
main.py	Waveform and log-Mel figures
asr.py	Wav2Vec2 CTC transcription
whisper_asr.py	Whisper transcription
ctc_demo.py	Illustrative greedy CTC collapse
wer.py	Two-model WER and CSV export
tts.ps1	Windows speech synthesis
stutter_speech_pipeline.ipynb	Notebook/Colab workflow
report/generate_pdf.py	Two-page mini-report generator

Run order (PowerShell)

```
Set-Location "D:\stutter project 191"
$py = ".\.venv\Scripts\python.exe"
& $py main.py
& $py asr.py
& $py whisper_asr.py
& $py wer.py
.\tts.ps1
```

Model checkpoints may download on first run. WER results should be regenerated only after verifying the reference file against the source recording.

Conclusions, limitations, and next steps

Responsible interpretation and suggested improvements

Conclusions

The project demonstrates acoustic visualization, two contrasting ASR approaches, CTC decoding, WER calculation, and TTS. Its outputs make the pipeline concrete and reproducible. The current run shows different hypotheses from Wav2Vec2 and Whisper, but current WER values are not independently validated.

Limitations

- Single short recording; results cannot be generalized to other speakers or conditions.
- Reference transcript has not been independently verified and is biased toward Whisper.
- ASR may normalize or omit disfluencies; transcript conventions affect WER.
- Greedy CTC does not use a language-model beam search in this implementation.
- TTS output is synthetic and not a reconstruction of the original voice.

Priority next steps

1. Create and independently review a verbatim reference transcript.
2. Define a consistent disfluency and punctuation normalization policy.
3. Re-run WER and report edit counts with the verified reference.
4. Expand evaluation to more recordings and report per-speaker and aggregate results.
5. Preserve consent, privacy, and clear separation between technical outputs and clinical claims.

Selected references

Graves, A. et al. (2006). Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks. ICML.

Baevski, A. et al. (2020). wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. NeurIPS.

Radford, A. et al. (2023). Robust Speech Recognition via Large-Scale Weak Supervision. ICML.

JiWER documentation. Word error rate and alignment measures for automatic speech recognition evaluation.